Doc. no.:	P2174R1
Date:	2022-4-15
Audience:	EWG
Reply-to:	Zhihao Yuan <zy at miator dot net>

Compound Literals

This paper proposes standardizing an existing practice, namely compound literals, available in GCC, Clang, and EDG for C++ language modes. It gives code that uses the C compound literals equivalent semantics in C++.

Motivation

Allow forming anonymous arrays with an intuitive syntax

The author found that APIs that take references to arrays are not uncommon.

```
template<class charT, std::integral T>
void update_catalog(T const (&ident)[4], charT const *path);
```

More specifically, it is sometimes used as a replacement to `std::initializer_list<T>` , but with a static bound.

Usually, you can call APIs like this with a simple braced initializer.

```
update_catalog({3, 6, 0, 999}, "/some/path");
```

But in some cases, you may have to designate an element type to the array. The following code doesn't compile.

```
// error: deduced conflicting types for parameter 'T'
update_catalog({3, 6, 0, BIG_LIT}, "/some/path");
```

Nor does

```
// [unintelligible error message to express "no such grammar"]
update_catalog(unsigned []{3, 6, 0, BIG_LIT}, "/some/path");
```

But since it's just a grammar issue, an intuiative fix would be – see if I can group the tokens:

```
update_catalog((unsigned []){3, 6, 0, BIG_LIT}, "/some/path");
```

And you got compound literals, currently supported in GCC, Clang, and EDG-based compilers such as NVCC and Intel. It enables us to form anonymous arrays quite easily – a lot easier than a workaround,

```
update_catalog(std::type_identity_t<unsigned[]>{3, 6, 0, BIG_LIT}, "/some/path");
```

albeit with differences in semantics, which will be discussed later.

Allow more C and C++ code to interoperate and ease migration

Macros that expand to compound literals are found often in C headers:

```
#define NN_VENDORID(x) ((nn_vendor_id_t){ 0x01, NN_VENDORID_MINOR##x })
```

Sometimes they come with designated initializers as well. Libraries with a C heritage tend to benefit more and more from the two features at the same time in design:

```
VkResult result = vkCreateInstance(
    &(VkInstanceCreateInfo){
        .sType = VK_STRUCTURE_TYPE_INSTANCE_CREATE_INFO,
        .pApplicationInfo =
            &(VkApplicationInfo){
                .sType = VK_STRUCTURE_TYPE_APPLICATION_INFO,
                .pApplicationName = "Hello World",
                .applicationVersion = VK_MAKE_VERSION(0, 1, 0),
                .pEngineName = "No Engine",
                .apiVersion = VK_API_VERSION_1_2
            }},
    NULL, &instance);
```

Since adopting C++20 designated initializers, we now have a certain level of support for the initializer macros. Adding C++ compound literals should be able to complete further the picture of sharing headers and make the code fluent when using those modern C libraries in C++.

Stop punishing C++ programmers with knowledge of C

From experts to ordinary users, many people believe that compound literals are a part of C++ because many C++ compilers, except MSVC, support the proposed syntax. We occasionally find open-source contributors reverting uses of compound literals in people's code.

Wasting time isn't the most harmful outcome of leaving the compound literals non-standard. The implementation divergencies between compilers can threaten the correctness of a program.

Recall that compound literals in C produce objects with scope lifetime. While in C++, if evaluating an expression gives you an anonymous object as the result, the object is destroyed at the end of the full-expression. The following chart shows the mess when implementations adopt compound literals as vendor-specific language extensions to C++.

Compiler	**Value category**	**Can take address**	**Type with dtor**	**Type without dtor**
GCC	prvalue	No	Temporary object	Possibly scope lifetime
Clang	prvalue	Only via array-to-pointer conversion	Temporary object	Scope lifetime
EDG	lvalue in local scope, prvalue in namespace scope	Yes	Temporary object	Scope lifetime

The C++ standard should give a clear answer to end this situation that can be learned from no book and prevent security risks from being introduced into the programs just because their authors learned more.

Design Decisions

This paper proposes closely matching C++ compound literals semantics with C's by making them

- **produce lvalue**, and

- support only **trivially destructible** types.

Given the fact that a typedef followed by *braced-init-list* produces prvalue in C++,

```
using arr_t = double[];
auto&& x = arr_t{ 3, 4, 5 };  // double(&&)[3]
```

It may create some surprises if adding a pair of parentheses changes the expression's value category.

```
using arr_t = double[];
auto&& x = (arr_t){ 3, 4, 5 }; // double(&)[3]
```

However, it's not news in C, where casts create rvalues. It's not news to the C++ committee, either, as braced initialization chose the different syntax to prevent assigning subtly different semantics to the C syntax.[1] Discussion of R0 of the paper in SG22 concluded that doing so can minimize the breakage among existing compilers and serve the target users sufficiently well.

With this decision, C++ users will be able to enjoy some new practices. For example, you can safely use buffer-returning APIs without naming a buffer.

```
char *ptr = strcat((char [100]){0}, "like this");
```

Wording

The wording is relative to N4910.

Extend the grammar:

> *cast-expression*:
>
> *unary-expression*
>
> (*type-id*) *cast-expression*
>
> (*type-id*) *braced-init-list*

Insert a new paragraph after [expr.cast]/1 (http://eel.is/c++draft/expr.cast#1):

> The result of the expression `(T)` *cast-expression* is of type `T`. The result is an lvalue if `T` is an lvalue reference type or an rvalue reference to function type and an xvalue if `T` is an rvalue reference to object type; otherwise the result is a prvalue. *[Note*: […] — *end note]*
>
> If an expression is of form `(` *type-id* `)` *braced-init-list*, let `init` be the *braced-init-list* and `T` be the *type-id*. `T` shall be a non-class type, a class type with a trivial destructor, or an array thereof. The expression introduces a variable with a unique name `e`
>
> ```
> T e init;
> ```
>
> and is an lvalue that refers to `e`.

Acknowledgments

Thank Aaron Ballman, Charlie Barto, and JeanHeyd Meneide for providing valuable feedback that reshaped this paper.

References

1. Stroustrup, Bjarne and Gabriel Dos Reis. N2215 *Initializer lists (Rev. 3)*. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2215.pdf (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2215.pdf) ↩